



CÓDIGO LIMPO

FUNDAMENTOS E LEGIBILIDADE BÁSICA

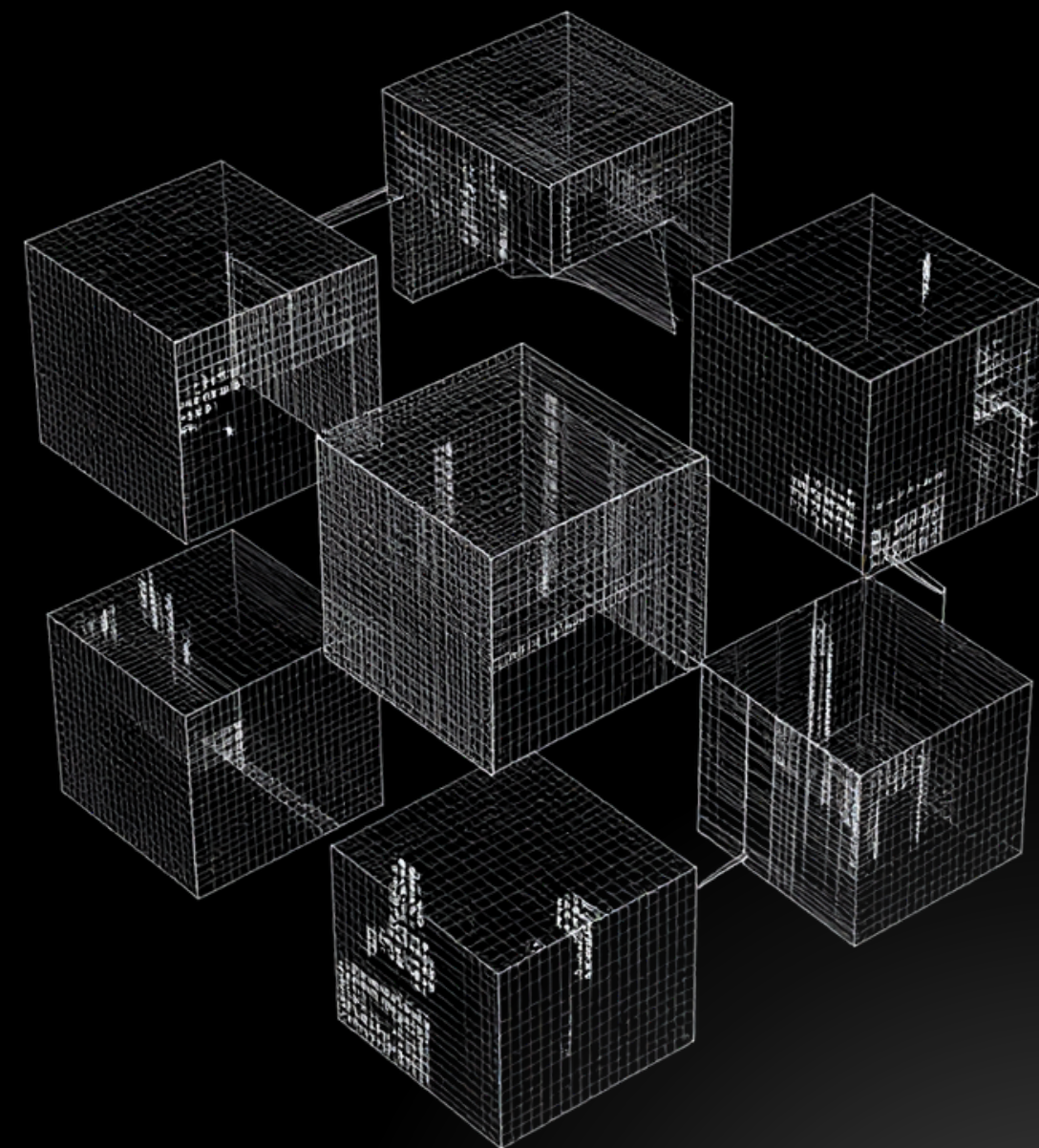
GRUPO 1

Felipe Rodrigo
Lucas Braz
Valter Sérgio
Felipe Rodrigues
Joshua

O QUE É UM CÓDIGO LIMPO

"Gosto do meu código elegante e eficiente. A lógica deve ser direta para dificultar o encobrimento de bugs, as dependências mínimas para facilitar a manutenção, o tratamento de erros completo de acordo com uma estratégia clara e o desempenho próximo do mais eficiente, de modo a não incitar pessoas a tornarem o código confuso com atualizações sorrateiras, um código limpo faz bem apenas uma coisa"

BJARNE STROUSTRUP



CUSTO DE UM CÓDIGO SUJO

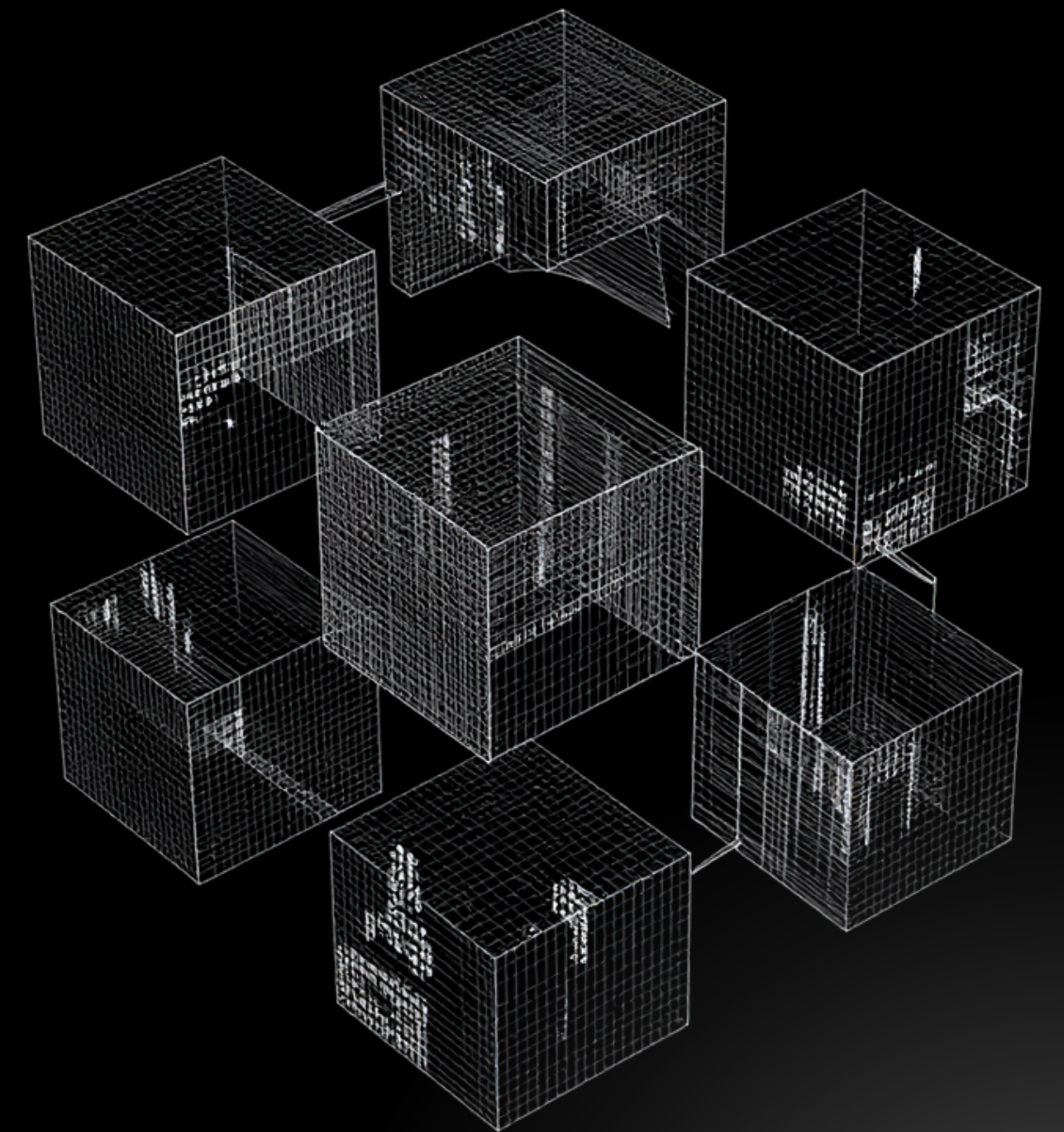
"USAR UM CÓDIGO RUIM É COMO NAVEGAR EM UM LAMAÇAL DE ARBUSTOS COM ARMADILHAS OCULTAS"

Um código ruim abre brechas e margens para alterações que ao invés de aprimorarem o código, diminuem sua produtividade, obrigando os programadores a perderem tempo com remendos e amarrações que poderiam ser evitadas toda vez que uma nova alteração é feita, ate que não sobre outra alternativa a não ser iniciar do começo.

REGRA DO ESCOTEIRO

"SEMPRE DEIXE O CÓDIGO MAIS LIMPO DO QUE VOCÊ ENCONTROU"

Um código precisa ser mantido sempre limpo para que ele não se degrade com o tempo (Deixe a área do acampamento mais limpa do que como você encontrou). A limpeza não precisa ser algo grande, trocar o nome de uma variável, dividir uma função que esteja um pouco grande, eliminar um pouco de repetição, reduzir uma instrução if aninhada são atitudes que simplesmente melhora o código com o tempo, o aperfeiçoamento continuo te faz um profissional melhor

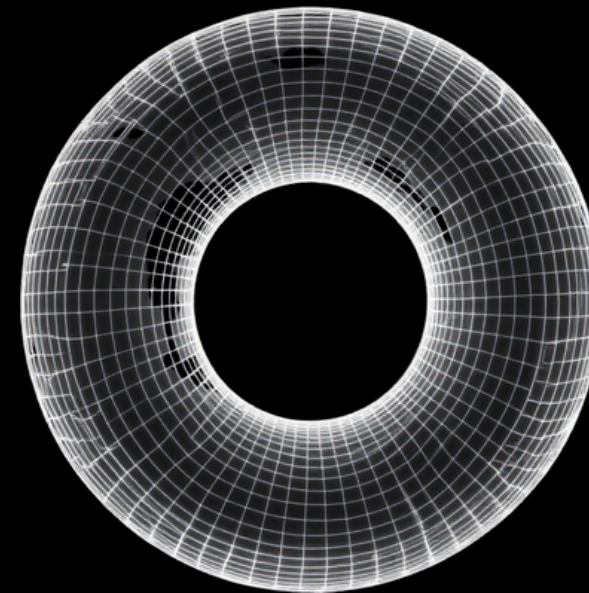


NOMES SIGNIFICANTES



Usar nomes pronunciáveis e buscáveis

Nós programadores já lidamos com codificação demais, seus nomes de variáveis e funções não precisam ser mais um problema



Verbos e Substantivos

Os nomes de uma classe devem ser substantivos como: *Cliente, pagina, carro*
Já nomes de métodos devem ser atribuídos com verbos como: *SalvarCliente, DeletarPagina, LigarCarro*



Nomes reveladores e inconfundíveis

Conseguir entender do que se trata, sem precisar fazer um mapa mental do resto do código, eles devem ser claros e não podem ser mal interpretados como: *ListContas* (sendo que o tipo da variável é um array), *Mod* (Pode ser *Modificador, modulo, Operação matemática* dentre outras possibilidades), ou *N3* (E aí o leitor advinha que é a nota do terceiro bimestre ou a terceira nota)

EXEMPLOS

Código sujo

```
private static void Main(string[] args)
{
    int idAl01 = 17;
    int idAl02 = 19;
    int calcId;

    if (idAl01 == idAl02)
    {
        Console.WriteLine("Os alunos possuem idades iguais");
    }
    else
    {
        Console.WriteLine("Os alunos possuem idades diferentes");
    }
}
```

EXEMPLOS

Código limpo

```
public static void Main(string[] args)
{
    int idade_primeiro_aluno = 17;
    int idade_segundo_aluno = 19;
    if (compararIdadesIguais(idade_primeiro_aluno, idade_segundo_aluno)) {
        Console.WriteLine("Os alunos possuem idades iguais");
    }
    else { Console.WriteLine("Os alunos possuem idades diferentes"); } ;

}

1 reference
public static bool compararIdadesIguais(int idade_primeiro_aluno, int idade_segundo_aluno)
{
    return (idade_primeiro_aluno == idade_segundo_aluno);
}
```

EXEMPLOS

Int D / Int DiasDesdeCriação
Const BDK / Const ChaveBancoDados
List list1 / list ListaAlunos / list Alunos
Int Tempo / int TempoEsperaSegundos
String verifica / String UsuárioVerificado

Void Enviar(Usuário U)
/
Void EnviarEmailBoasVindas(Usuário
Usuário)

Int ObterIdade (Int id)/ Int CalcularIdade
(Int idUsuario)

FUNÇÕES

01

Tamanho

Funções não devem passar de 4 a 5 linhas maximas de conteudo

02

Função

Uma função deve fazer apenas uma coisa, e faze-la bem

03

0/1 Parâmetros

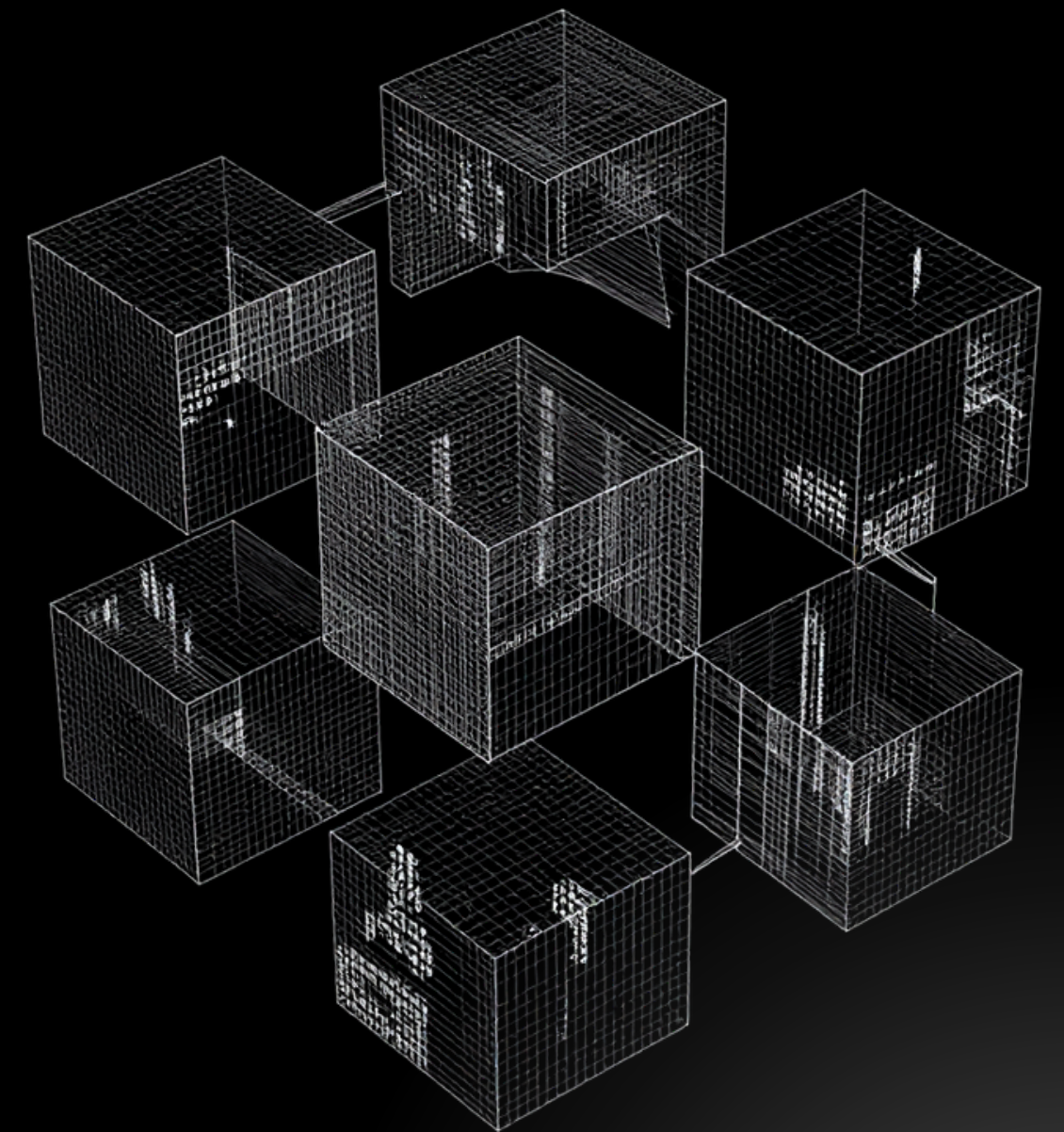
Priorize funções com um ou nenhum parâmetro, chamadas de funções nulas ou mônades

SEPARAÇÃO COMANDO-CONSULTA

VERBO OU ADJETIVO?

"As funções devem fazer ou responder a algo, mas nunca ambos"

Devemos focar em funções que ou alteram o estado sobre um objeto ou retornam informações sobre ele (exemplos claros disso são os getter e setter separados), dessa forma podemos evitar problemas de confusão e estrutura como o asseguir.



EXEMPLO

Públic boolean set(string atributo, string Valor) // Essa função define um valor a um atributo e retorna true caso consiga, e false caso não, o que de certa maneira significa fazer duas coisas

então em uma condição if ficaria assim:
If(set("Nome","Valter"))

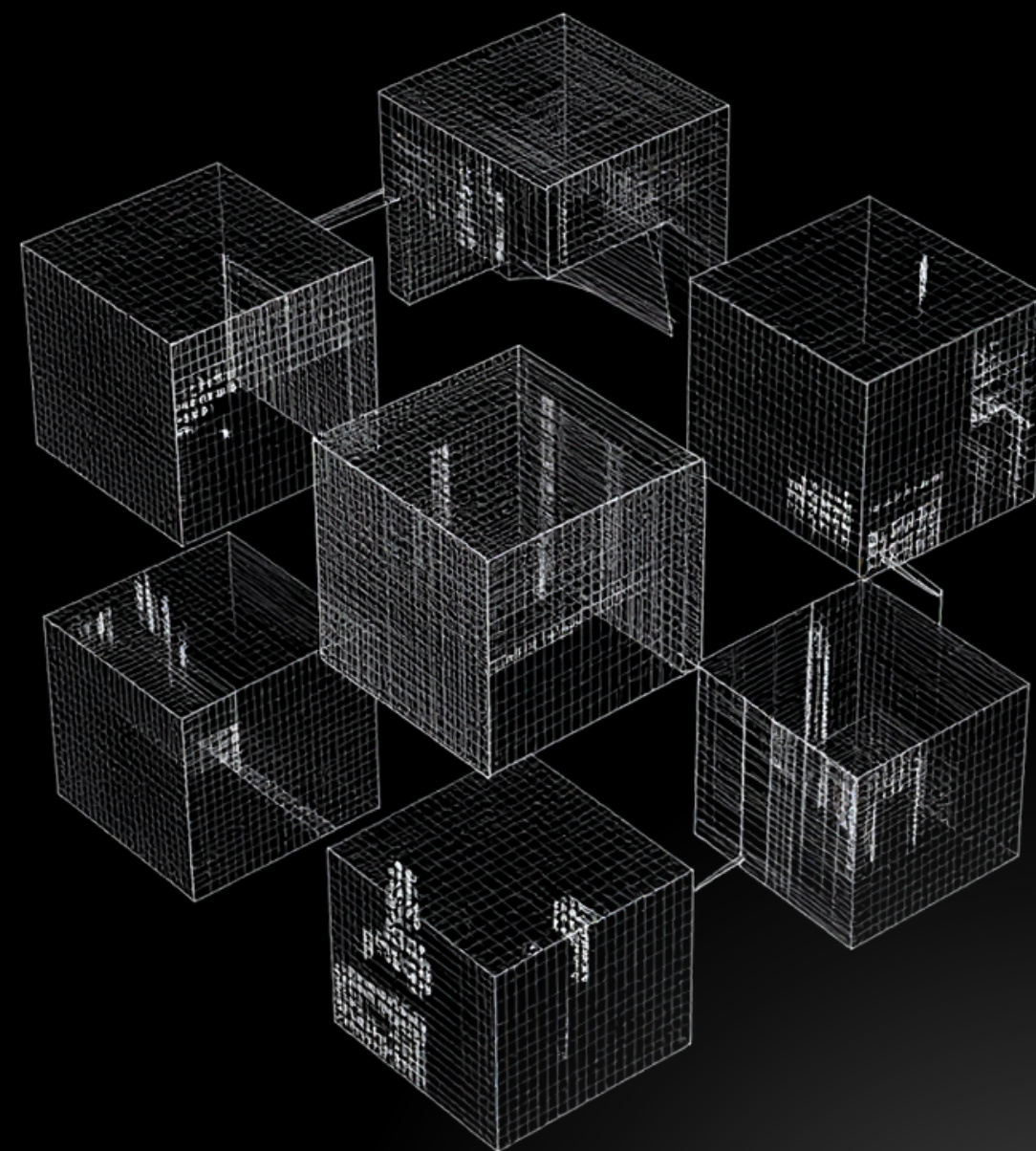
você não sabe se esta atribuindo o valor ou se esta verificando se aquele atributo possui aquele valor, um jeito de concertar isso seria:

```
if(AtributoExiste("Nome")) {  
    Set("nome","Valter")  
}
```

EFEITOS COLATERAIS

EFEITOS COLATERAIS SÃO UMA MENTIRA!

Quem afirmou isso foi o autor do livro, ao explicar que efeitos colaterais são quando uma função se compromete a fazer apenas uma coisa e acaba fazendo duas, como no exemplo anterior onde ocorria a validação de sucesso – mesmo que o nome da função fosse Set – e a definição de valor, tudo na mesma função, outro exemplo seria uma função de verificação de senha que abre o aplicativo caso esteja correta, isso gera problemas entre a equipe, que acredita no que a função diz a ele, e pode acabar criando funções que colidam uma com as outras.



CONCLUSÃO

Portanto, concluimos que escrever um código limpo se trata de cuidar de cada janela e maçaneta em sua casa por menor que seja, dirigindo a devida atenção aos detalhes, desta forma será mais fácil lidar com "insetos" que tentem atrapalhar sua produtividade através de confusão, repetição e sobrecarga.



OBRIGADO!

"SE FOR PRA FAZER DE QUALQUER JEITO,
ENTÃO NEM FAÇA"